深 圳 大 学 实 验 报 告

课程名称　Python 程序设计

项目名称　NumPy 实现 ViT 图像编码器

学　　院　人工智能学院

专　　业　人工智能专业

指导教师　樊超

报 告 人　余家铭　学号 2025681055

实验时间　2026 年 11 月 10 日周二

提交时间　2026 年 11 月 11 日周三

教务处制

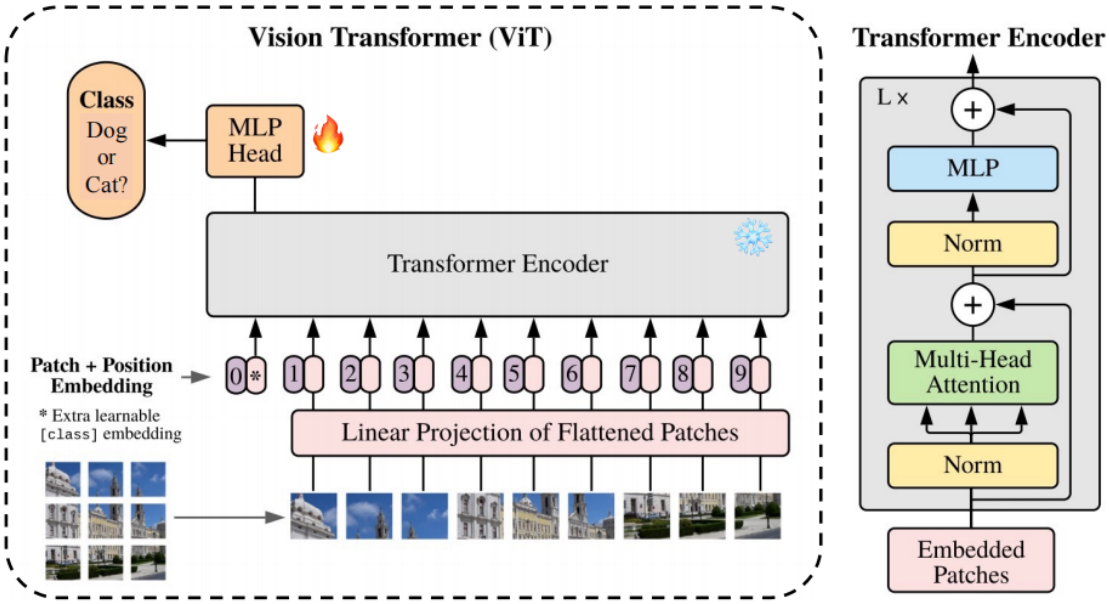


图 1: ViT 图像编码器的总体前向流程

1 实验目的

本项目的目标是使用 NumPy 完成 OpenCLIP ViT-B/32 图像编码器的前向计算。模型参数、图像预处理程序和参考结果由项目提供，不涉及模型训练。编码器接收形状为 $[B, 3, 224, 224]$ 的图像数组，经过图像块划分、token 嵌入、12 层 Transformer 和输出投影，得到每张图像对应的 512 维单位向量。

实验重点掌握 `reshape`、`transpose`、矩阵乘法和广播等 NumPy 操作；理解多头注意力、残差连接及 MLP 的前向过程。调试完成后，利用提供的独立 FP32 参考数组验证各阶段计算结果。

2 总体流程

程序的整体数据流如图 1 所示，主要步骤如下：

1. 将预处理后的图像按 32×32 划分为 $7 \times 7 = 49$ 个图像块，并将每个 RGB 图像块展平为 3072 维向量；
2. 将图像块投影到 768 维，添加 CLS token 和位置嵌入，形成 $[B, 50, 768]$ 的 token 序列；
3. 依次通过 12 个 Transformer 块，从最终序列中取出 CLS token，经后置归一化、$768 \to 512$ 投影和单位长度归一化得到输出。

3 实验过程

图像块划分与多头组织

实验先从图像块划分开始。我先根据输入形状确定每张图像应分成 7×7 个 32×32 图像块，再将 $[B, 3, 224, 224]$ 重排为 $[B, 3, 7, 32, 7, 32]$。为了让同一位置的 RGB 数据保留在同一个图像块中，需要按以下顺序转置各轴：

```
x = x.transpose(0,2,4,1,3,5)
```

这样两个图像块网格轴位于通道轴之前，最后展平为 $[B, 49, 3072]$。在调试时，我曾把通道轴合并到图像块数量中，结果却变成 147 个单通道小块。随后搞清楚模型要求，我将排列修正为 $[B, 7, 7, 3, 32, 32]$，才得到正确的 49 个 RGB 图像块。

多头部分采用同样的思路处理。我们将 token 的 768 维特征拆成 12 个注意力头，每个头包含 64 维特征。数组先由 $[B, N, D]$ 变成 $[B, N, H, D/H]$，再交换 token 轴与 head 轴，得到 $[B, H, N, D/H]$。

```python
def patchify(self, images):
        B = images.shape[0]
        x = images.reshape(B, 3, 7, 32, 7, 32)
        x = x.transpose(0, 2, 4, 1, 3, 5)
        return x.reshape(B, 49, 3072)
```

(a) TODO-01：图像块划分

```python
def split_heads(self, x):
        B, N, D = x.shape
        H = self.heads
        DdH = D // H
        res = x.reshape(B, N, H, DdH)
        res = res.transpose(0, 2, 1, 3)
        return res
```

(b) TODO-02：拆分多头

图 2: 图像块划分与多头形状变换的实现

注意力与 Transformer 块

完成拆头后，用点积缩放 $\frac{QK^{\mathsf{T}}}{\sqrt{d}}$ 计算形状为 $[B, H, N, N]$ 的注意力分数。实际的 K 是四维数组，因此我们只交换最后两个轴，保留 batch 和 head 的分组。分数经过提供的 `attention_weights` 转换为权重，再与 V 相乘得到各头的上下文。交换 head 与 token 轴，最后将 12 个头重新合并为 $[B, N, 768]$ 并完成输出投影。

Transformer 块中保留了两条残差路径。第一次将注意力更新量加回原输入；第二次对该结果进行 LayerNorm，再通过 $768 \to 3072 \to 768$ 的 MLP 和 GELU，最后将 MLP 更新量加回。



```python
def __call__(self, x, causal=False):
        qkv = x @ self.qkv_weight.T + self.qkv_bias
        q = self.split_heads(qkv[:, :, :self.width])
        k = self.split_heads(qkv[:, :, self.width:2*self.width])
        v = self.split_heads(qkv[:, :, 2*self.width:])

        scores = (q @ k.transpose(0, 1, 3, 2)) / np.sqrt(np.float32(self.head_dim))

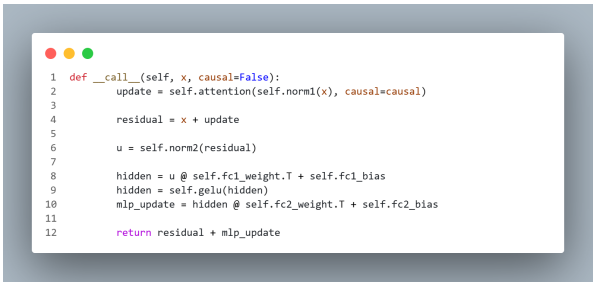
        if causal:
            scores = causal_mask(scores)
        coefficients = attention_weights(scores)  # supplied numerical helper

        B, N, D = x.shape

        context = coefficients @ v
        merged = context.transpose(0, 2, 1, 3).reshape(B, N, D)

        return merged @ self.out_weight.T + self.out_bias
```

(a) TODO-03：多头注意力



```python
def __call__(self, x, causal=False):
        update = self.attention(self.norm1(x), causal=causal)

        residual = x + update

        u = self.norm2(residual)

        hidden = u @ self.fc1_weight.T + self.fc1_bias
        hidden = self.gelu(hidden)
        mlp_update = hidden @ self.fc2_weight.T + self.fc2_bias

        return residual + mlp_update
```

(b) TODO-04：残差连接与 MLP

图 3: 注意力计算和 Transformer 块的实现

Token 嵌入与最终输出

图像块经过无偏置投影后得到 $[B, 49, 768]$，随后在 token 轴最前面拼接 CLS token，并通过广播加上位置嵌入，形成 $[B, 50, 768]$。在 12 层 Transformer 计算完成后，取第 0 个 CLS token，依次进行了后置 LayerNorm、$768 \to 512$ 投影和单位长度归一化。



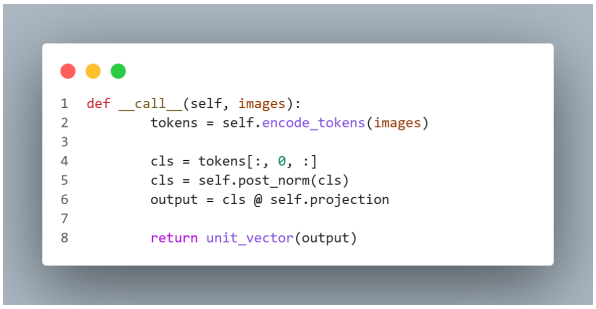
```python
def embed(self, images):
        patches = self.patchify(images)

        tokens = patches @ self.patch_weight.T

        cls = np.tile(self.cls, (images.shape[0], 1, 1))

        tokens = np.concatenate((cls, tokens), axis=1)
        return tokens + self.position
```

(a) TODO-05：token 嵌入



```python
def __call__(self, images):
        tokens = self.encode_tokens(images)

        cls = tokens[:, 0, :]
        cls = self.post_norm(cls)
        output = cls @ self.projection

        return unit_vector(output)
```

(b) TODO-06：CLS 输出

图 4: 输入 token 构造和最终输出的实现

表 1: 各阶段与参考数组的数值误差

阶段	输出形状	最大绝对误差	平均绝对误差
preprocess	$(10, 3, 224, 224)$	0.000e+00	0.000e+00
patches	$(2, 49, 3072)$	0.000e+00	0.000e+00
tokens	$(2, 50, 768)$	4.005e−05	1.289e−06
block0	$(2, 50, 768)$	7.063e−06	3.250e−07
last_tokens	$(2, 50, 768)$	4.768e−05	2.696e−06
output	$(10, 512)$	5.439e−07	8.311e−08
batch	$(3, 512)$	4.619e−07	7.632e−08

4 实验结果

在项目目录执行 `check.py`。图 5 是本次实际运行保存的终端截图。程序通过了资产、预处理、中间 token、最终输出和改变 batch 顺序等全部检查。误差均在合理范围内，因此我们认为实验复现是成功的。随后利用 `run.py` 导出 `output_tokens.npz`，成功保存形状为 $(10, 512)$ 的输出数组。

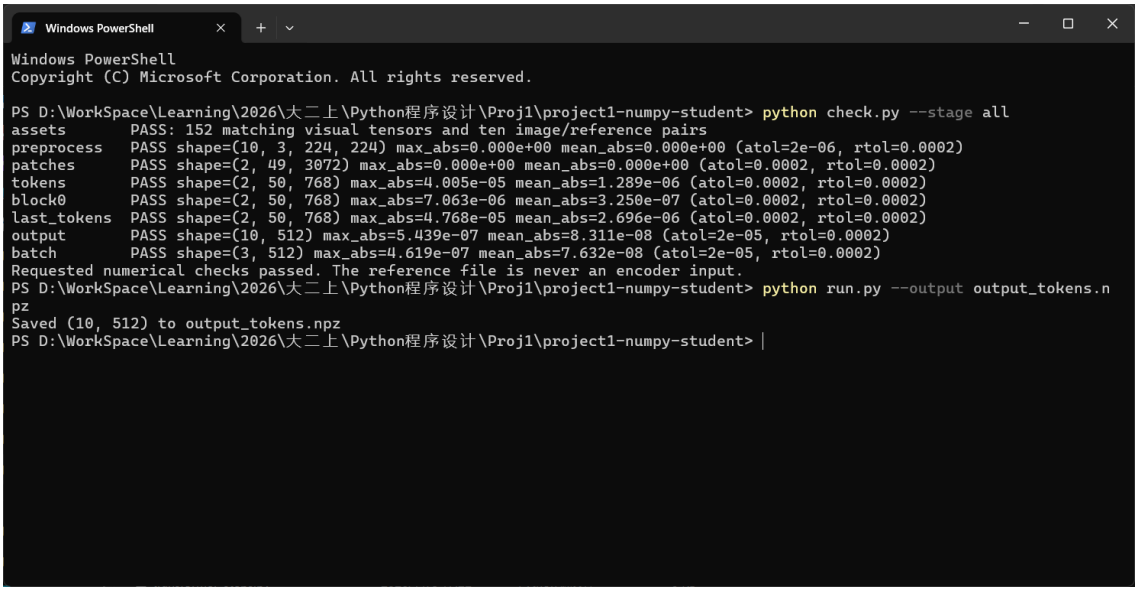


图 5: 完整数值检查及输出文件生成结果

5 分析与讨论

在划分图像块时，我曾把通道轴合并到图像块数量中，得到 147 个块，但这和要求的 49 个包含 RGB 三通道的图像块矛盾。因此我重排了维度顺序，才得到正确的 $[B, 49, 3072]$。这一问题说明我们要检查张量维度的排序和形状，同时确认每个轴代表的对象。

数值结果分析。预处理和 `patches` 阶段与参考结果完全一致，最大绝对误差均为 0。内部阶段允许的绝对误差为 2×10^{-4}，实际最大误差不超过 4.768×10^{-5}；最终 `output` 阶段允许的绝对误差为 2×10^{-5}，实际最大误差为 5.439×10^{-7}，约为该阈值的 $1/37$。所有阶段均通过形状、数据类型、有限值和 `np.allclose` 检查，这说明了实现与独立 FP32 参考结果高度一致，我们可以认为实验成功复现了。`batch` 检查还改变了图像顺序并包含重复图像，结果仍通过，说明程序能够处理不同顺序和重复输入，而不是依赖固定顺序返回结果。

参考结果评估。 `check.py` 会读取初始给出的期望输出参数，分别计算当前实现的中间结果和最终输出，再用 `np.allclose` 比较。参考数组仅用于检查，不会作为编码器输入。

AI 辅助说明。本实验使用 GPT 6 Astra 辅助调试，并帮助理解深度学习和数值计算相关概念，协助完成 LaTeX 排版。核心实验、代码和测试由个人完成。

指导教师批阅意见：

成绩评定：

指导教师签字：

年　　月　　日

备注：

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。